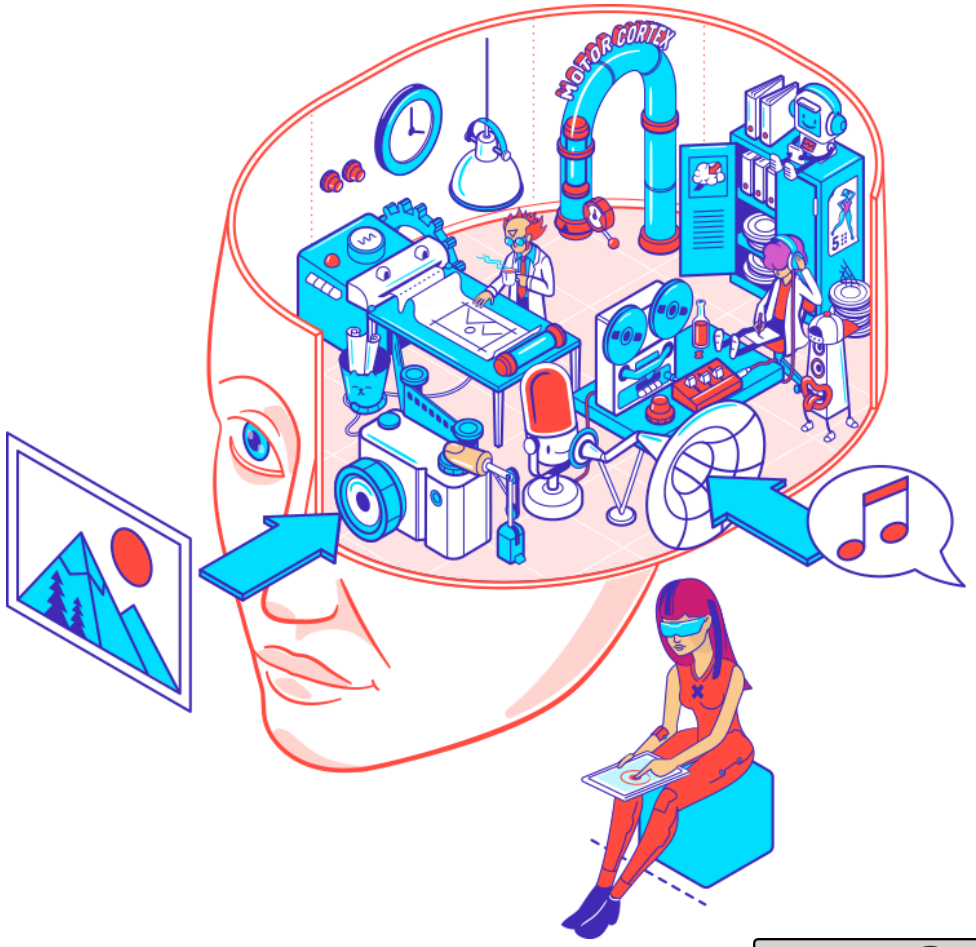
HAW HAMBURG

MEDIA PROGRAMMING

UNIT 6: INTERACTIVE GRAPHICS



Prof. Dr. Eike Langbehn

Department of Media Technology
Faculty of Design, Media and Information
Hamburg University of Applied Sciences

CC BY

AGENDA:

1. Introduction
2. From Java to Python
3. Raster Graphics
4. Algorithmic Graphics Generation
5. Vector Graphics
6. **Interactive Graphics**
7. Animations

LEARNING OUTCOME

- Understanding **principles** of interactive graphics
- Knowledge of **how to implement** interactive graphics

BASICS

UNIT 6: INTERACTIVE GRAPHICS



INTERACTIVE GRAPHICS

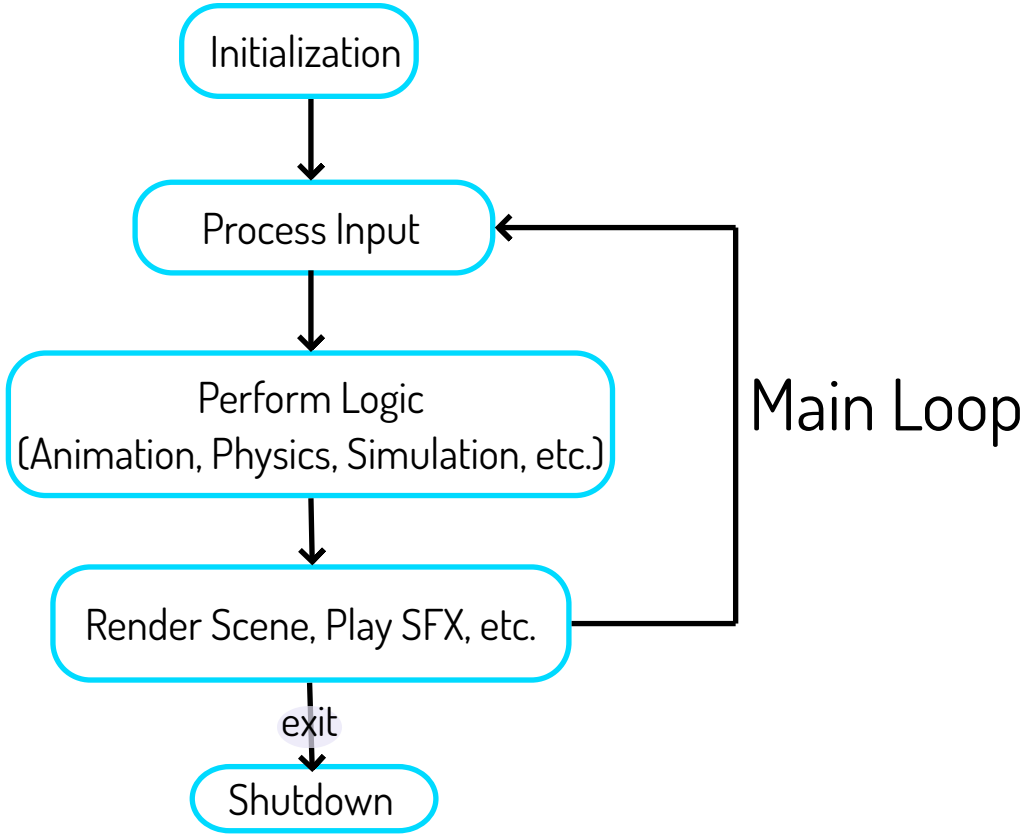
Definition

Interactive graphics refers to computer graphics systems where the **user can influence the display in real time**, creating a dynamic feedback loop between human and machine.

Static Graphics	Interactive Graphics
Image is computed	Image is computed & reacts on input
Static output	Dynamic output
Pre-rendered	Real-time Rendering
One-time algorithm	Continuously updated
Picture	"Running" application

MAIN LOOP

Main Loop is the central construct of every interactive real-time system

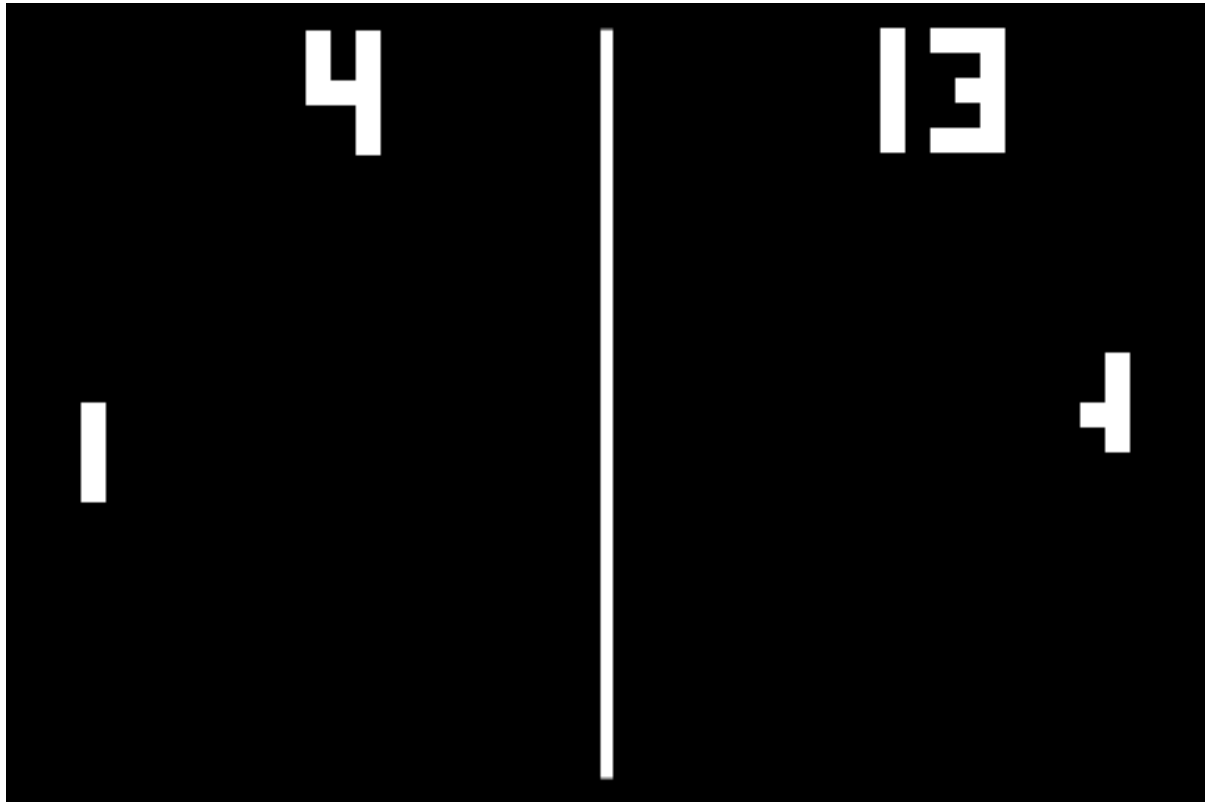


Framerate (FPS)

The speed at which the **Main Loop** completes one full iteration determines the Frames Per Second. To maintain 60 FPS, the entire loop (Input → Logic → Render) must finish in under **16.6ms**.

EXAMPLE: PONG

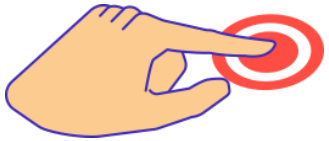

```
1  void main() {
2      initGame();
3      while (true) { // game loop
4          readHumanInterfaceDevices();
5          if (quitButtonPressed())
6              break; // exit the game loop
7              movePaddles();
8              moveBall();
9              collideAndBounceBall();
10             if (ballImpactedSide(LEFT_PLAYER)) {
11               incremenentScore(RIGHT_PLAYER);
12               resetBall();
13             } else if (ballImpactedSide(RIGHT_PLAYER)) {
14               incrementScore(LEFT_PLAYER);
15               resetBall();
16         }
17         renderPlayfield();
18      }
19 }
```

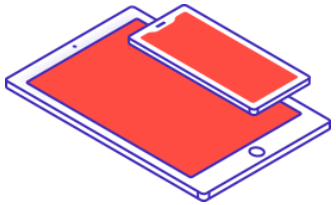


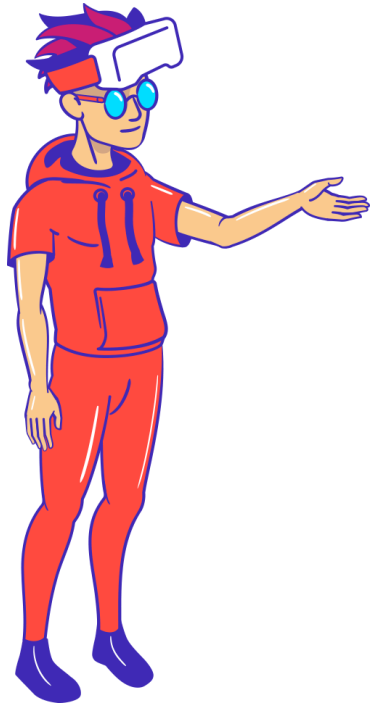
INPUT

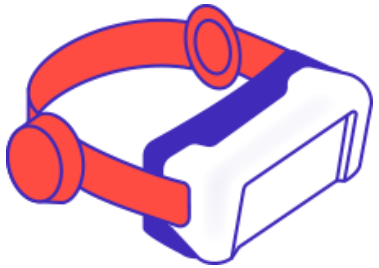
- Process user input
  - Mouse/Keyboard/Gamepad
  - Camera-Based (Kinect)
  - Rotational / Translational Tracking
  - Touch-based
  - Speech-based
  - ...

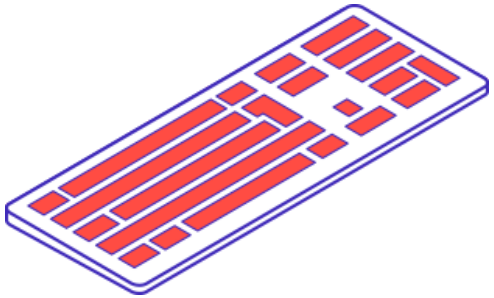












LOGIC

- Framerate problem: Object moves 2cm each update loop cycle
  - 30 FPS: 60cm moved in one second
  - 60 FPS: 120cm moved in one second
- Solutions:
  - Time-dependent movement (variable or fluid time step)
  - Fixed time step/frame rate (e.g. 30 FPS)
    - Variable time step is not deterministic cause of floating point numbers rounding errors
    - Important for physics, networking and AI

Example: Delta Time Calculation

```python
import time

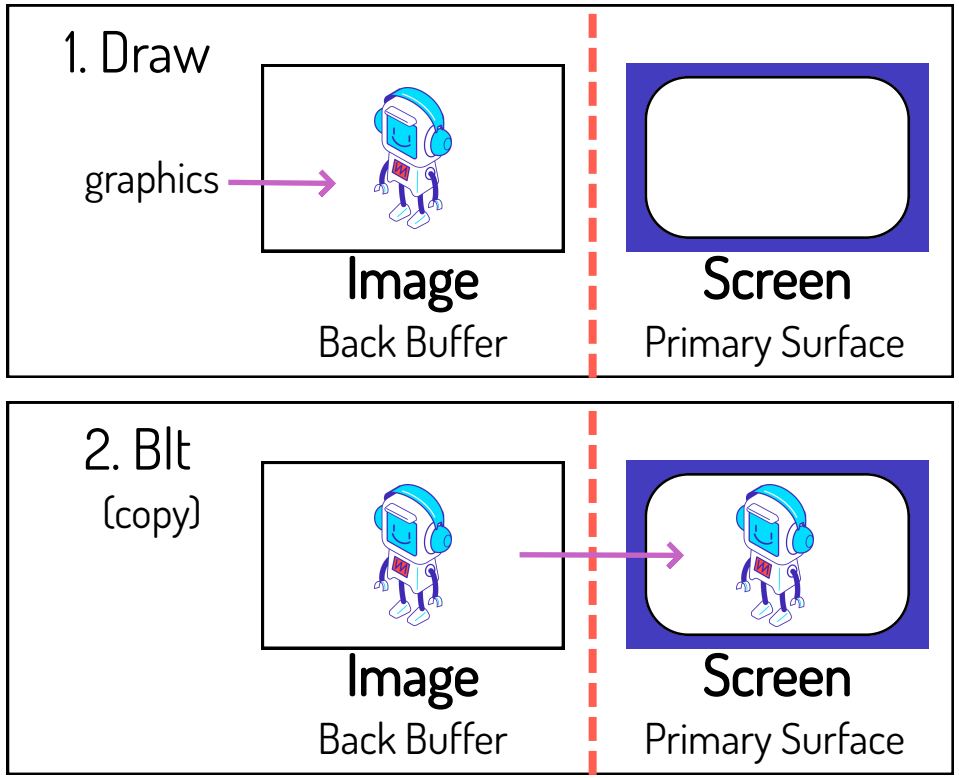
# Store the time of the last frame
last_time = time.time()

while True:
    # Calculate how much time has passed
    current_time = time.time()
    delta_time = current_time - last_time
    last_time = current_time

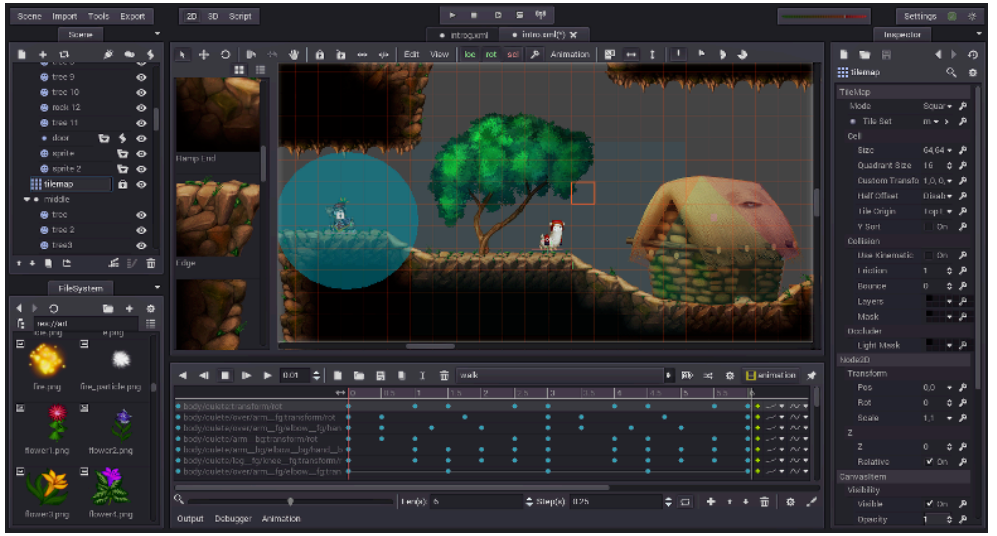
    # Use delta_time for consistent movement
    # speed * delta_time = distance per second
    object_x += speed * delta_time
```

RENDER

Double Buffering



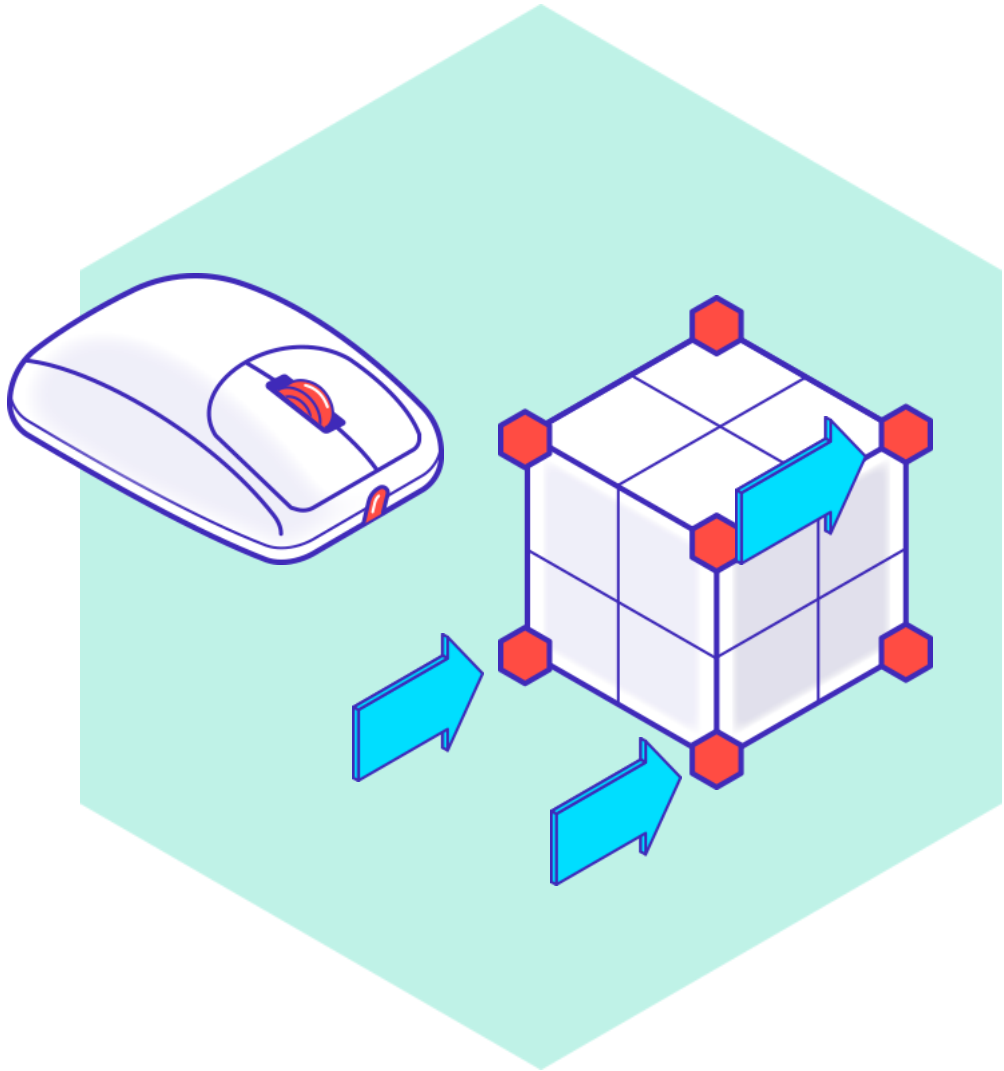
GODOT GAME ENGINE



- Open Source / Free
- Modern & flexible architecture (node-based)
- Uses a Python-like script language
- Extract and run. Godot is self-contained and does not require installation.
- Only 30 MB
- Editor runs on Win, Mac, Linux and **Web**
- Export to Win, Mac, Linux, iOS, Android, HTML5

THE GODOT EDITOR

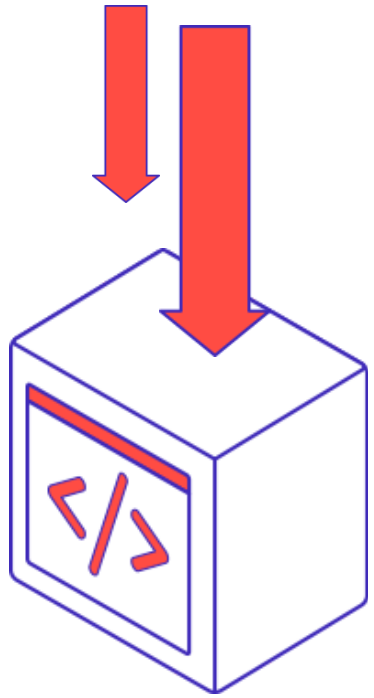
UNIT 6: INTERACTIVE GRAPHICS



UP AND RUNNING
DOWNLOAD & INSTALLATION

Follow the instructions to download and install the Godot Editor on the official Godot website.

You do not need to install the Export templates to start making games.



OVERVIEW OF THE GODOT EDITOR

- Visually edit game levels
- Build User Interface layouts for menus, overlays, inventories
- Change behavior of scripts for e.g. players, enemies, and items
- Animate in-game Objects and UI
- Write GDScript programs
- Read and search the Godot Manual

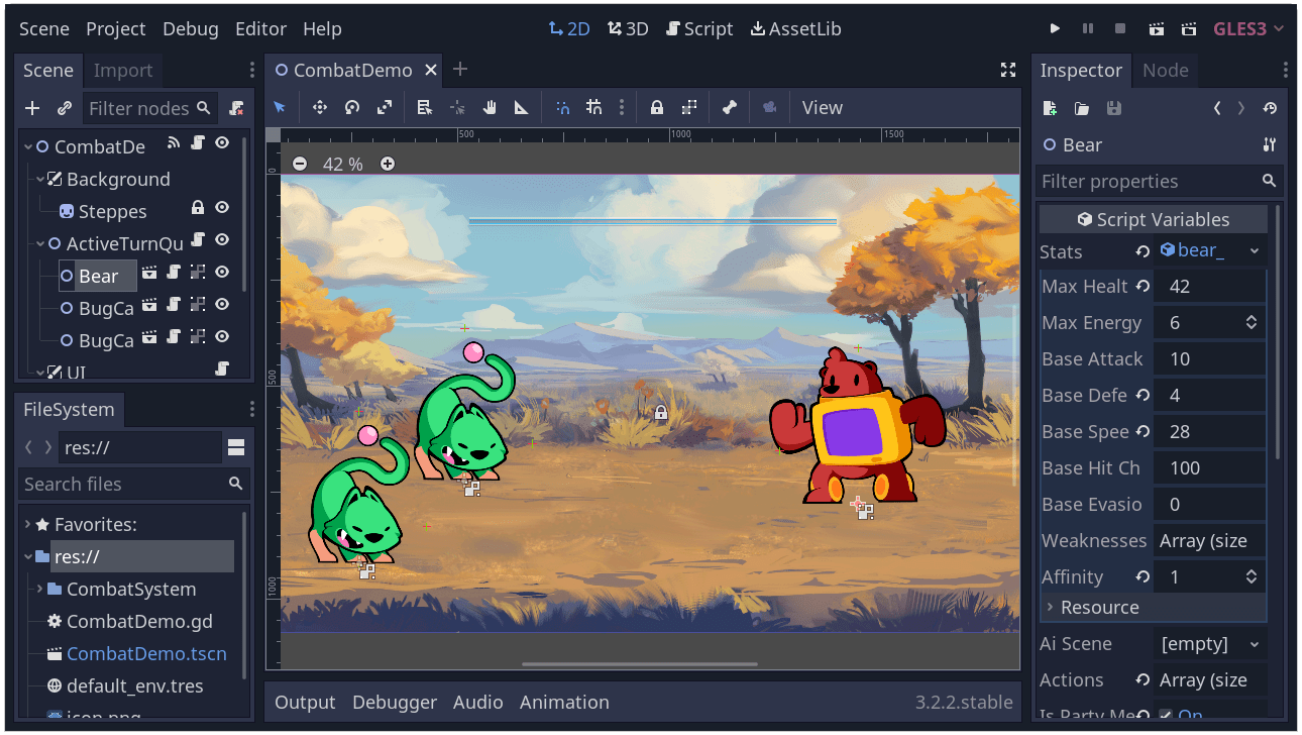


Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

THE GODOT EDITOR INTERFACE

- Main Menus:
  Settings, common actions
- Workspaces:
  2D Viewport, 3D Viewport, Scripting, Asset Library
- Playtest buttons:
  Start / pause / stop game or single *Scenes*
- Scene Tree Dock:
  Tree of *Nodes* (game objects) in the current *Scene*

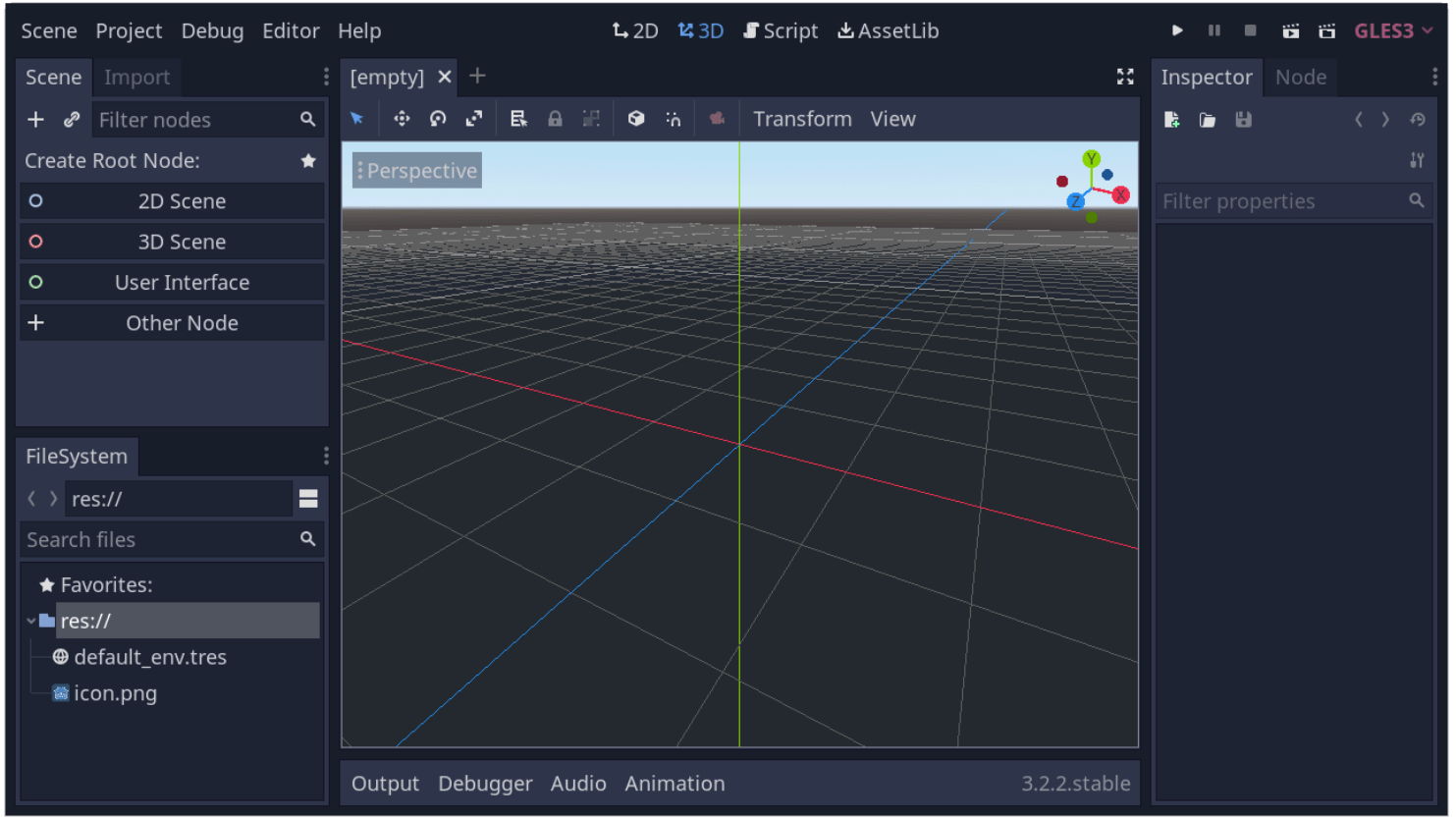


Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

THE GODOT EDITOR INTERFACE / 2

- Viewport:
  Level layout
- Toolbar:
  Manipulate objects in the viewport
- File System:
  Browse project files ("Assets")
- Inspector:
  Edit properties of *Nodes*
- Bottom Panel:
  Game logs, audio & animation docks, debugger

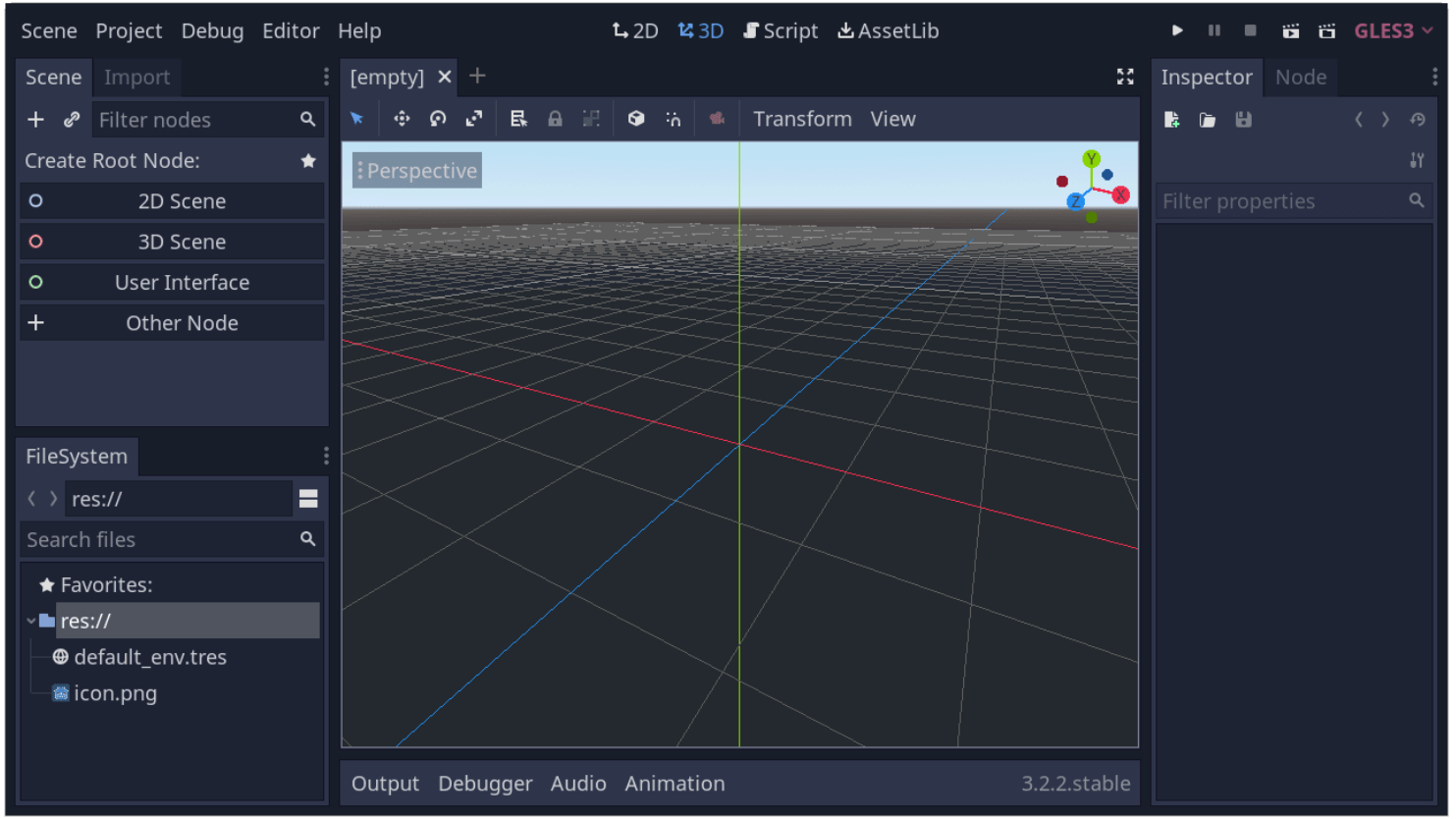


Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

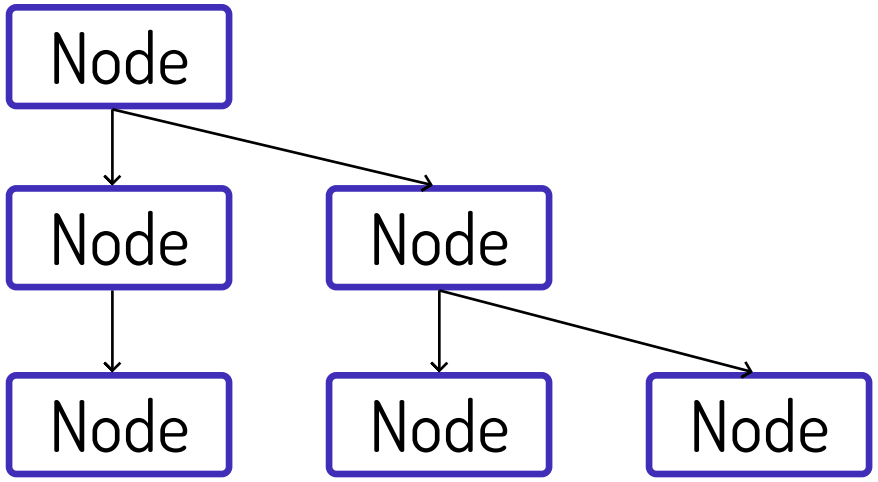
NODES AND SCENES

UNIT 6: INTERACTIVE GRAPHICS



NODES

- Have a name
- Have editable properties
- Can have scripted behavior
- Can have children
- Provide lots of functionality: Rendering graphics, playing sounds, simulating physics, etc.



SCENES

- Are groups of nodes, organized in a hierarchy
- Always have exactly one root node
- Can be saved in and loaded from a file
- Provide structure for the game
- Can include other scenes
- Are displayed and edited in the godot editor



A "Player" Scene in the Godot editor, with various nodes.

"Mannequiny" and "CameraRig" are sub-scenes of the "Player" Scene.

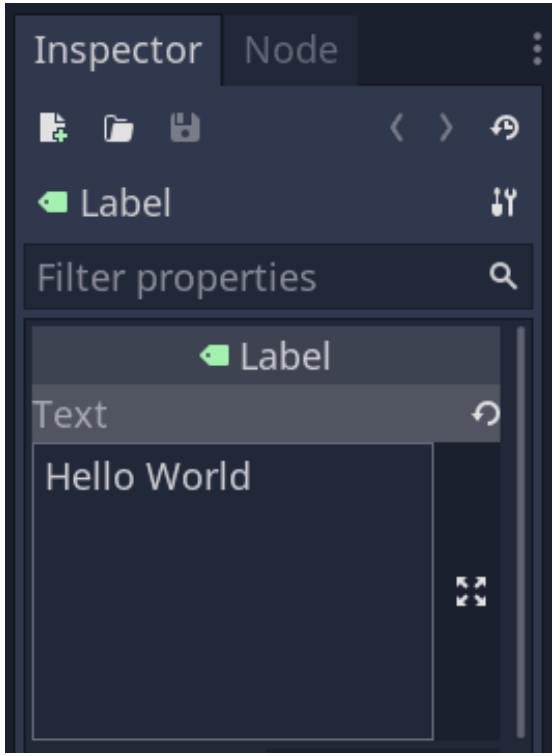
Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

NODES IN THE GODOT EDITOR

- Have a name
- Have editable properties
- Can have scripted behavior
- Can have children
- Provide lots of functionality out of the box: Rendering graphics, playing sounds, simulating physics, etc.

For a full list of built-in nodes, see Godot's Class Reference.

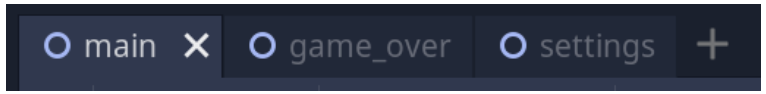




Left Image, Right Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

SCENES IN THE GODOT EDITOR

- Godot games are structured using many Scenes
- The editor can edit multiple scenes at once, e.g. a specific level, the game over screen, the settings menu, etc.
- The Viewport shows a preview of the currently selected scene
- Scenes can be switched using the tabs at the top of the editor window



Tabs in the Godot editor showing three scenes

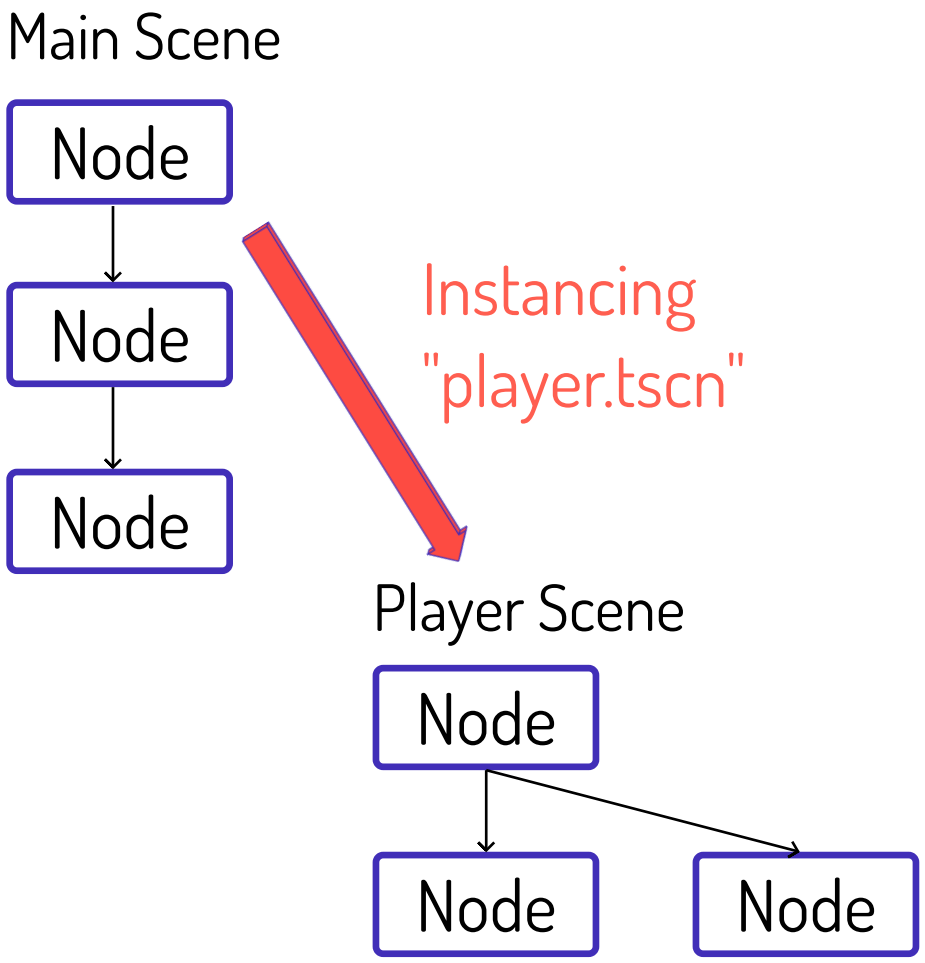
SCENES IN THE GODOT EDITOR / 2

- Every game has a main scene, which can be launched using the "Play" button
- The main scene is configured in the Project Settings
- Other scenes can be played independently of the main scene using the "Play Scene" and "Play Custom Scene" buttons



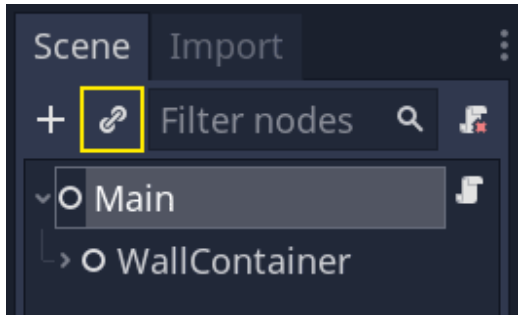
INSTANCING SCENES

- Scenes are saved to disk during development, then *loaded* and *instanced* when the game runs
- Saved scenes have a ".tscn" extension and are called *packed scenes*
- *Instancing* includes a packed scene in another scene as if it was a normal node



INSTANCING SCENES / 2

- The same scene can be instanced multiple times, e.g. to spawn enemies in a level
- Properties such as an enemy position have default values, but can be overridden for each instance of a scene



- The "instance" editor button adds another scene as a child to the current scene.

Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

SCRIPTING WITH GDSCRIPT

UNIT 6: INTERACTIVE GRAPHICS



ADDING BEHAVIOR TO NODES BY SCRIPTING

- Scripting adds *behavior* to nodes
- Godot supports scripting in many languages, e.g. C#, C++, VisualScript and GDScript
- GDScript is specifically built for Godot and integrates seamlessly with the engine
- The following slides contain an introduction to GDScript
- Each node can have exactly one script attached
- Scripting is *object-oriented*: each script is a class that inherits behavior from other classes

GDSCRIPT BASICS

- GDScript syntax is similar to python
- No "class" keyword is needed: Each GDScript file implicitly defines its own class
- "extends" defines the parent class, making methods and fields of the "Sprite" parent class available
- "var" introduces a variable. Variables at the top of a script become fields of the script's class

```
1  extends Sprite
2
3
4
5
6
7
8
9
10
```

GDSCRIPT BASICS

- "func" defines a function
- Virtual functions like "_init" and "_process" are inherited from the "Node" class and are overridden in our child class
- "_init" will be called by the engine once the node is created

```
1  extends Sprite
2
3  var speed = 400
4  var angular_speed = PI
5
6  func _init():
7      print("Hello, world!")
8
9
10
```

GDSCRIPT BASICS

- "_process" will be called by the engine on every frame, allowing us to update the fields of our class
- "rotation" is a field inherited from "Sprite", modifying it will make our sprite turn a little every frame

```
1  extends Sprite
2
3  var speed = 400
4  var angular_speed = PI
5
6  func _init():
7      print("Hello, world!")
8
9  func _process(delta):
10     rotation += angular_speed * delta
```

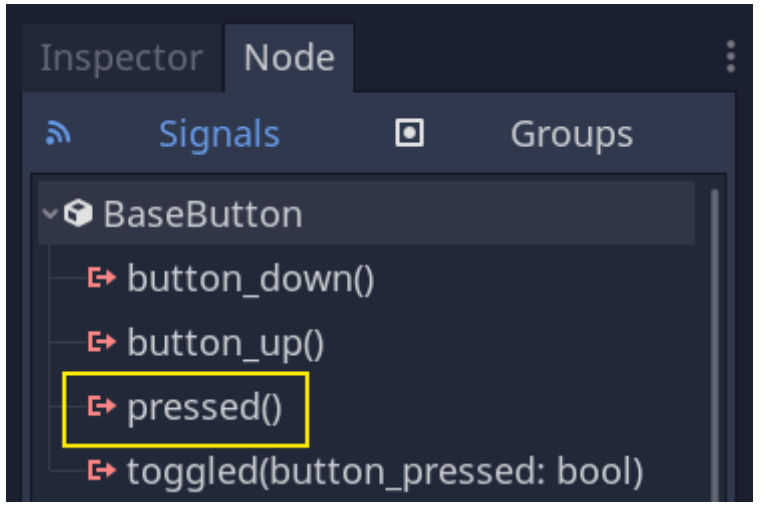
COORDINATING NODES

UNIT 6: INTERACTIVE GRAPHICS



NODE COMMUNICATION USING SIGNALS

- Signals allow nodes to be notified when something happens, e.g. when a button is clicked, the player pressed a key, ...
- Each node as its own signals that it can *emit*, and other nodes can *connect* with these signals
- A *connection* consists of a source signal and a destination function: "When *this* signal is emitted, call *this* function on *this* node"



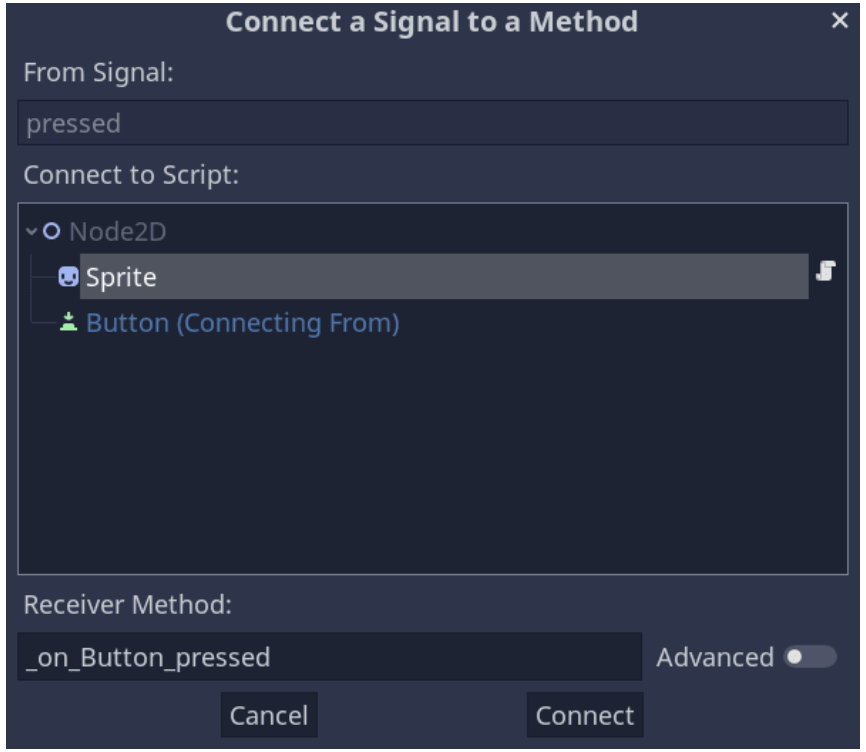
The list of signals provided by a "Button" node, shown in the editor

Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

CONNECTING SIGNALS IN THE EDITOR

- Double clicking a signal in the editor opens the node connection window
- Each connection consists of a source signal and a destination method on a specific node

- Connections created in the editor will be indicated beside the source code of the targeted method




```
23  func _on_Button_pressed():
24      pass # Replace with function body.
25
```

Images by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

CONNECTING SIGNALS IN SCRIPTS

- Example: In the script on the Sprite node, use the Timer child node to get notified after a certain amount of seconds has passed
- In the Sprite script, use the get_node function to get a reference to the Timer child node:

```
1 func _ready():
2     var timer = get_node("Timer")
```

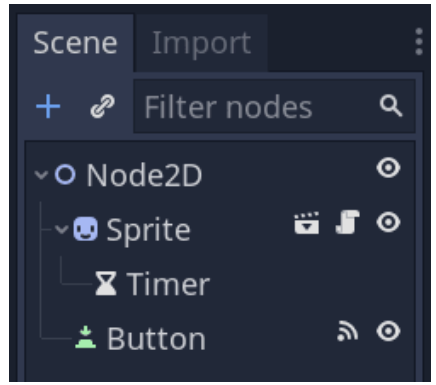


Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

CONNECTING SIGNALS IN SCRIPTS

- Use the connect function with three parameters:
  - "timeout": name of source signal
  - *self*: reference to the node implementing target method (*self* being the Sprite node itself)
  - "_on_Timer_timeout": name of target method

```
1 func _ready():
2     var timer = get_node("Timer")
3     timer.connect("timeout", self, "_on_Timer_timeout")
4
5 func _on_Timer_timeout():
6     visible = not visible
```

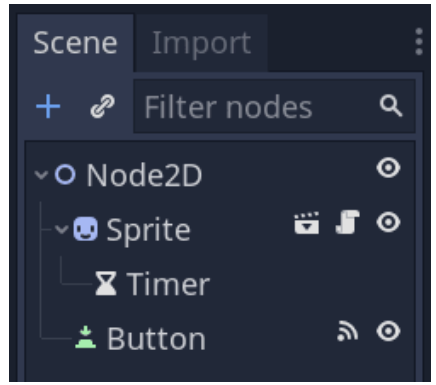


Image by Juan Linietsky, Ariel Manzur and the Godot community, licensed under CC-BY-3.0

GROUPING NODES

- get_node is used to find single nodes relative to the node it's called from
- To find multiple similar nodes, e.g. all enemies, organize them using groups

```
1  # in enemy.gd
2
3  func _ready():
4      add_to_group("enemies")
5
6  # in player.gd
7
8  func respawn():
9      for enemy in get_nodes_in_group("enemies"):
10         enemy.set_target(self)
```

Example: when the player respawns, make all enemies walk towards them

PLAYER INPUT

UNIT 6: INTERACTIVE GRAPHICS



CHECKING WHICH BUTTONS ARE PRESSED

- To check if certain mouse or keyboard buttons are pressed at the time of calling, use the global Input object
- This is useful for continuous input, e.g. mouse movements, joysticks, and WASD-style keyboard movement

```
1 func _process(delta):
2     var direction = 0
3     if Input.is_action_pressed("ui_left"):
4         direction = -1
5     if Input.is_action_pressed("ui_right"):
6         direction = 1
7
8     # move with a speed of 20 pixels per second
9     position.x += 20 * direction * delta
```

Example: check the left and right arrow buttons each frame

REACTING TO BUTTON TAPS

- Use the _unhandled_input method to get notified when buttons are pressed or released
- This is useful for input that is discrete, e.g. buttons that are tapped or mouse clicks

```
1 func _unhandled_input(event):
2     # When the player jumps, instantly
3     # move them 100 pixels up in the air
4     if event.is_action_pressed("jump"):
5         position.y += 100
```

Example: when the player taps the jump button, move them into the air

LEARNING MORE AND GETTING HELP

Godot Documentation

This should be your first stop for anything you do not understand. Every part of Godot is explained here.

Godot Q&A

A StackOverflow-like forum. Try searching for existing questions!

Godot Discord Server

Chat with other Godot users.

GDQuest tutorials and examples

Videos and example projects explaining Godot concepts, tips and tricks.

Resource links in the Godot Documentation

Links to more interesting tutorials and tools.

Godot Asset Library

Lots of useful tools and examples available directly from the Godot editor.

HAW HAMBURG

Unless otherwise noted, the contents of this Open Educational Resource are licensed under the Creative-Commons Attribution 4.0 International license (CC BY 4.0)
Attribution: Rafael Epplée, Eike Langbehn, Anh Sang Tran, Peter Wood

